DAT 471 – Computational Techniques for Large-Scale Data

Module 7

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

28 May 2026

Problem 1 - Batching

Part a

New implementation with batching:

```
def linear_scan(X, Q, b = None):
    m = Q.shape[0]
    I = np.zeros(m,dtype=np.int64)

    batch_size = b if b is not None else m

    for i in range(0,m,batch_size):
        current_batch = Q[i:i+batch_size]

        distances = np.linalg.norm(current_batch[:, None, :] -
         X[None, :, :], axis=2)

        I[i:i+batch_size] = np.argmin(distances, axis=1)

    return I
```

For correctness verification the implementation was used on the pubs dataset with the tiny quey version, resulting in:

```
Loading dataset (46988 vectors of length 2) took 0.2150123119354248
Performing 10 NN queries took 0.0234832763671875
Number of erroneous queries: 0
```

Part b

The following tests were all executed on the query sizes of tiny, small and medium for consistency considering the large computational time for *Glove.840B.300d*.
Running the original code results:
pubs

Tiny

```
Loading dataset (46988 vectors of length 2) took 0.2954990863800049
Performing 10 NN queries took 1.1583938598632812
Number of erroneous queries: 0
```

Small

```
Loading dataset (46988 vectors of length 2) took 0.08329319953918457
Performing 100 NN queries took 6.340791940689087
Number of erroneous queries: 0
```

Medium

```
Loading dataset (46988 vectors of length 2) took 0.14518141746520996
Performing 1000 NN queries took 62.23570275306702
Number of erroneous queries: 0
```

Glove.60B.50d
tiny

```
Loading dataset (400000 vectors of length 50) took 2.6605660915374756
Performing 10 NN queries took 5.490191698074341
Number of erroneous queries: 0
```

small

```
Loading dataset (400000 vectors of length 50) took 2.2067084312438965
Performing 100 NN queries took 53.905709743499756
Number of erroneous queries: 0
```

medium

```
Loading dataset (400000 vectors of length 50) took 1.6833720207214355
Performing 1000 NN queries took 587.303787946701
Number of erroneous queries: 0
```

Glove.840B.300d
tiny

```
Loading dataset (2196017 vectors of length 300) took 108.13088059425354
Performing 10 NN queries took 32.52477049827576
Number of erroneous queries: 0
```

small

```
Loading dataset (2196017 vectors of length 300) took 101.58324527740479
Performing 100 NN queries took 329.3430690765381
Number of erroneous queries: 0
```

medium

```
Loading dataset (2196017 vectors of length 300) took 101.13656377792358
Performing 1000 NN queries took 3230.4588990211487
Number of erroneous queries: 0
```

Running batch implementation:
Different batch sizes where tested on each individual dataset using the small query size. Best result was used for the rest of the query sizes.

pubs
Tested batched sizes:

- 100
- 500
- 1000

Best result acquired from size 100.

tiny

```
Loading dataset (46988 vectors of length 2) took 0.16486620903015137
Performing 10 NN queries took 0.010283231735229492
Number of erroneous queries: 0
```

Small

```
Loading dataset (46988 vectors of length 2) took 0.1620326042175293
Performing 100 NN queries took 0.13060641288757324
Number of erroneous queries: 0
```

Medium

```
Loading dataset (46988 vectors of length 2) took 0.10455060005187988
Performing 1000 NN queries took 0.8956139087677002
Number of erroneous queries: 0
```

Glove.60B.50d
Bigger dataset + bigger batch size → much more memory usage. Therefore tested batch sizes:

- 10
- 50
- 100

Best result acquired from size 10:

tiny

```
Loading dataset (400000 vectors of length 50) took 1.8583440780639648
Performing 10 NN queries took 3.727979898452759
Number of erroneous queries: 0
```

Small

```
Loading dataset (400000 vectors of length 50) took 2.22286319732666
Performing 100 NN queries took 32.05080270767212
Number of erroneous queries: 0
```

Medium

```
Loading dataset (400000 vectors of length 50) took 1.9678142070770264
Performing 1000 NN queries took 326.01899099349976
Number of erroneous queries: 0
```

Glove.840B.300d
Batches above 10 significantly increased the execution time of the program, therefore the batch test were set to:

- 1
- 3
- 5

Best result acquired from size 3.

tiny

```
Loading dataset (2196017 vectors of length 300) took 131.16032338142395
Performing 10 NN queries took 250.17297410964966
Number of erroneous queries: 0
```

Small

```
ets/glove/glove.840B.300d_queries_small_names.txt -b 3
Loading dataset (2196017 vectors of length 300) took 103.89334988594055
Performing 100 NN queries took 1356.7184617519379
```

Medium

```
Loading dataset (2196017 vectors of length 300) took 132.67210292816162
Performing 1000 NN queries took 22126.587713479996
Number of erroneous queries: 0
```

For the dataset Glove.840B.300d, running it with batching took considerably longer than the original implementation. This could be due to the dataset being both large and high-dimensional. The memory required grows with the data size, its dimensionality and the batch size. The cost increase is due to the memory managment,its movement and the implemented overhead. This shows the tradeoffs of batching.

Dataset	Query size	Throughput
Pub	10	971 queries/s
Pub	100	766 queries/s
Pub	1000	9560 queries/s
glove50	10	3 queries/s
glove50	100	3 queries/s
glove50	1000	3 queries/s
glove300	10	0.04 queries/s
glove300	100	0.07 queries/s
glove300	1000	0.05 queries/s

Table 1: Throughput of program

Problem 2 - CuPy Version

Part a

To convert the implementation of problem 1 into a GPU version some minor changes were made to the *linear_scan* function, as can be seen in the following:

```
def linear_scan(X, Q, b = None):
    m = Q.shape[0]
    I = cp.zeros(m,dtype=cp.int64)

    X_gpu = cp.asarray(X)
    Q_gpu = cp.asarray(Q)

    batch_size = b if b is not None else m

    for i in range(0,m,batch_size):
        current_batch = Q_gpu[i:i+batch_size]

        distances = cp.linalg.norm(current_batch[:, None, :]
        - X_gpu[None, :, :], axis=2)

        I[i:i+batch_size] = cp.argmin(distances, axis=1)

    cp.cuda.Stream.null.synchronize()

    return cp.asnumpy(I)
```

Part b

For consistency the same batch size, for each respective dataset, was taken from previous problem.

pub -b 100
tiny

```
Time to transfer data to GPU: 0.00020360946655273438
GPU computation time: 1.2035737037658691
Time to transfer results to CPU: 0.00012612342834472656
Loading dataset (46988 vectors of length 2) took 0.18595123291015625
Performing 10 NN queries took 1.4365243911743164
Number of erroneous queries: 0
```

small

```
Time to transfer data to GPU: 0.00019311904907226562
GPU computation time: 1.3203775882720947
Time to transfer results to CPU: 0.00014138221740722656
Loading dataset (46988 vectors of length 2) took 0.17598438262939453
```

```
Performing 100 NN queries took 1.5688722133636475
Number of erroneous queries: 0
```

medium

```
Time to transfer data to GPU: 0.00020837783813476562
GPU computation time: 1.3906738758087158
Time to transfer results to CPU: 0.00012564659118652344
Loading dataset (46988 vectors of length 2) took 0.2069385051727295
Performing 1000 NN queries took 1.6358845233917236
Number of erroneous queries: 0
```

Glove.60B.50d

tiny

```
Time to transfer data to GPU: 0.07530450820922852
GPU computation time: 1.3896541595458984
Time to transfer results to CPU: 0.00013327598571777344
Loading dataset (400000 vectors of length 50) took 2.555973529815674
Performing 10 NN queries took 1.698875904083252
Number of erroneous queries: 0
```

small

```
Time to transfer data to GPU: 0.07550358772277832
GPU computation time: 1.3080098628997803
Time to transfer results to CPU: 0.0001494884490966797
Loading dataset (400000 vectors of length 50) took 2.5738747119903564
Performing 100 NN queries took 1.6079630851745605
Number of erroneous queries: 0
```

medium

```
Time to transfer data to GPU: 0.07602953910827637
GPU computation time: 1.788437843322754
Time to transfer results to CPU: 0.00011587142944335938
Loading dataset (400000 vectors of length 50) took 2.552495002746582
Performing 1000 NN queries took 2.0783438682556152
Number of erroneous queries: 0
```

Glove.840B.300d

tiny

```
Time to transfer data to GPU: 1.3916707038879395
GPU computation time: 3.8589541912078857
Time to transfer results to CPU: 0.00024366378784179688
Loading dataset (2196017 vectors of length 300) took 47.69373035430908
Performing 10 NN queries took 5.454608678817749
Number of erroneous queries: 0
```

small

```
Time to transfer data to GPU: 1.3914401531219482
GPU computation time: 2.407578706741333
Time to transfer results to CPU: 0.0001049041748046875
Loading dataset (2196017 vectors of length 300) took 46.14512062072754
Performing 100 NN queries took 3.9972221851348877
Number of erroneous queries: 0
```

medium

```
Time to transfer data to GPU: 1.3822147846221924
GPU computation time: 13.57416296005249
Time to transfer results to CPU: 0.00010085105895996094
Loading dataset (2196017 vectors of length 300) took 47.90762782096863
Performing 1000 NN queries took 15.152998208999634
Number of erroneous queries: 0
```

Problem 3 - Matrix Multiplication

Part a

To avoid the memory and runtime cost of explicit pairwise distance computations, we use the identity:

$$\|x - q\|^2 = \|x\|^2 + \|q\|^2 - 2\langle x, q\rangle.$$

Let $X \in \mathbb{R}^{n\times d}$ be the dataset and $Q \in \mathbb{R}^{m\times d}$ the query matrix. We first compute the squared norms:

$$X_{\text{sq}} = \|x_i\|^2 \in \mathbb{R}^{n\times 1}, \qquad Q_{\text{sq}} = \|q_j\|^2 \in \mathbb{R}^{1\times m},$$

and the inner–product matrix:

$$XQ^\top \in \mathbb{R}^{n\times m}.$$

Using broadcasting, the full squared distance matrix is:

$$D = X_{\text{sq}} + Q_{\text{sq}} - 2XQ^\top.$$

The nearest neighbor for each query is obtained by taking the columnwise arg min of D. This method is significantly faster than the loop-based implementation because it relies on optimized BLAS matrix multiplication. However, it is less numerically stable in single precision, especially for high-dimensional data, since the expression involves subtracting large, similar quantities.

Part b — Empirical Evaluation of the Matrix–Multiplication NN Implementation (CPU, Chunked)

We evaluated the CPU-based matrix–multiplication nearest neighbour implementation on all three datasets (Pubs 2D, GloVe 50D, GloVe 300D) and for all query sizes (tiny, small, medium, big). To avoid excessive memory usage, the implementation processes the dataset in chunks, ensuring that even the largest dataset (GloVe 300D, 2.19M vectors) can be evaluated without exhausting system memory.
For each experiment we report:

- total runtime,
- throughput (queries per second),
- number of incorrectly reported nearest neighbours.

All experiments were executed inside the Apptainer container.

Pubs Dataset (2D, 46,988 vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	0.0394	254.04	1	10
small	0.0526	1899.86	18	100
medium	0.4105	2435.91	145	1000
big	46.1522	216.67	1657	10000

Table 2: CPU matrix–multiplication results for the Pubs dataset.

As expected for a low-dimensional dataset, numerical instability is visible: the number of incorrect neighbours increases with query size due to floating–point cancellation in 32-bit arithmetic.

GloVe 6B (50D, 400,000 vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	0.0645	154.94	0	10
small	0.2594	385.51	0	100
medium	2.8237	354.15	0	1000
big	387.6276	25.80	0	10000

Table 3: CPU matrix–multiplication results for the GloVe 50D dataset.

The 50-dimensional dataset exhibits excellent numerical stability: all query sets produce zero incorrect nearest neighbours. Runtime increases significantly for the big query set due to repeated matrix–multiplication over large chunks.

GloVe 840B (300D, 2.19M vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	10.7304	0.93	0	10
small	12.1588	8.22	0	100
medium	24.2450	41.25	0	1000
big	1980.2134	5.05	0	10000

Table 4: CPU matrix–multiplication results for the GloVe 300D dataset.

Chunking enables all query sizes to run successfully, including the big case, which previously failed due to memory exhaustion. Despite the dataset's size, all query sets achieve perfect correctness.

Conclusion

Across all datasets:

- The Pubs dataset shows increasing numerical errors due to floating–point cancellation in low dimensions.
- Both GloVe datasets (50D and 300D) exhibit perfect numerical stability.
- Chunking allows the matrix–multiplication method to scale to multi-million point datasets without memory failures.
- Runtime grows substantially with query size, especially for high-dimensional data, but throughput remains consistent with the cost of repeated matrix–multiplication.

Overall, the chunked CPU matrix–multiplication implementation is correct, memory-safe, and capable of handling even the largest dataset in the assignment.

Problem 4 - Matrix Multiplication on the GPU

Part a - GPU Implementation

The nearest–neighbour algorithm from Problem 3 was reimplemented to run on the GPU using CuPy. The method follows the same distance identity,

$$\|x - q\|^2 = \|x\|^2 + \|q\|^2 - 2\,x^\top q,$$

but performs all operations on the CUDA device. The implementation includes:

- transferring the dataset and queries to the GPU using `cp.asarray` with blocking enabled,
- computing squared norms and batched matrix multiplications on the GPU,
- selecting nearest neighbours using `cp.argmin`,
- synchronizing the CUDA stream before timing,
- transferring results back to the CPU with `cp.asnumpy`.

A small warm-up transfer is performed to initialize the CUDA context, and queries are processed in batches to avoid excessive memory usage. Additionally, the program detects whether the GPU is available; if CUDA initialization fails (e.g., due to driver/runtime mismatch), it automatically falls back to the CPU implementation to ensure that all experiments can still run.

Part b - Evaluation of the GPU Implementation

We evaluated the GPU–accelerated linear scan on all datasets and all query sizes. The implementation uses CuPy with two–level batching (query batching and dataset chunking), which eliminates GPU out-of-memory failures even for the largest GloVe dataset.

Pubs Dataset (2D)

The pubs dataset contains 46,988 points in $\mathbb{R}^2$. Table 5 summarizes the GPU runtimes and correctness.

Query set	Queries	Runtime (s)	Throughput (q/s)	Errors
tiny	10	4.4923	2.23	1
small	100	0.5700	175.44	18
medium	1000	0.8176	1223.11	143
big	10000	0.6061	16498.83	1648

Table 5: GPU results for the pubs dataset.

The GPU achieves substantial speedups as the number of queries grows, reaching over 1.6×10^4 queries/s for the largest batch. The number of incorrect neighbors increases with query count, reflecting the low dimensionality and many near-ties in the dataset.

GloVe 50D Dataset

The GloVe 50D dataset contains 400,000 vectors in $\mathbb{R}^{50}$. GPU performance is shown in Table 6.

Query set	Queries	Runtime (s)	Throughput (q/s)	Errors
tiny	10	0.6102	16.39	0
small	100	0.6563	152.37	0
medium	1000	0.7170	1394.78	0
big	10000	1.0721	9327.42	0

Table 6: GPU results for the GloVe 50D dataset.

All queries return correct nearest neighbors. Throughput increases with batch size, reaching over 9,000 queries/s.

GloVe 300D Dataset

The largest dataset contains 2,196,017 vectors in $\mathbb{R}^{300}$. Chunking is essential here to avoid GPU memory exhaustion. Results are shown in Table 7.

Query set	Queries	Runtime (s)	Throughput (q/s)	Errors
tiny	10	1.7720	5.64	0
small	100	1.7985	55.60	0
medium	1000	2.1656	461.77	0
big	10000	6.0445	1654.39	0

Table 7: GPU results for the GloVe 300D dataset.

Despite the dataset's size, the GPU processes up to 10,000 queries in under 6.1 seconds with perfect correctness. This demonstrates that chunked GPU linear scan scales effectively to multi-million point, high-dimensional datasets.

Conclusion

Across all datasets:

- GPU throughput increases dramatically with query batch size.
- The pubs dataset shows non-zero error counts due to many near-ties in 2D; both GloVe datasets achieve perfect accuracy.
- Chunking is required for the 300D dataset to avoid out-of-memory errors; with chunking, all experiments complete successfully.

Overall, the GPU implementation provides substantial speedups and scales to the largest dataset without memory failures.

Problem 5 - CUVS

Part a - Implementation

The implementation is straightforward: the dataset X and query matrix Q are transferred to the GPU as `cupy` arrays, and the CUVS routine is invoked directly. The function returns both distances and indices; we extract the index of the nearest neighbour for each query.

Part b - Empirical Evaluation of the GPU Brute-Force Baseline

In this section, we evaluate the GPU brute-force nearest neighbour baseline implemented using CuPy with chunking. Chunking is required because the full distance matrix does not fit in GPU memory for the larger datasets (GloVe 50D and GloVe 300D).
The implementation computes exact nearest neighbours and serves as the ground-truth baseline for comparison with the approximate methods.
For each dataset and query set we report:

- total runtime,
- throughput (queries per second),
- number of incorrect nearest neighbours compared to the reference,
- dataset loading time.

All experiments were executed on an NVIDIA L4/L40s GPU inside the assignment container.

Pubs Dataset (2D, 46,988 vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	3.3118	3.02	1	10
small	0.5018	199.27	18	100
medium	0.4792	2086.64	145	1000
big	0.5440	18381.53	1657	10000

Table 8: GPU brute-force results for the Pubs dataset.

GloVe 6B (50D, 400,000 vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	0.5584	17.91	0	10
small	0.5531	180.81	0	100
medium	0.6195	1614.14	0	1000
big	1.0718	9330.04	0	10000

Table 9: GPU brute-force results for the GloVe 50D dataset.

GloVe 840B (300D, 2.19M vectors)

Query set	Runtime (s)	Throughput	Errors	Queries
tiny	1.8958	5.27	0	10
small	1.9040	52.52	0	100
medium	2.2520	444.05	0	1000
big	5.7274	1746.00	0	10000

Table 10: GPU brute-force results for the GloVe 300D dataset.

Discussion

The GPU brute-force baseline achieves exact nearest neighbour search across all datasets and query sizes. The chunked implementation enables evaluation even on the largest dataset (2.19M vectors in 300 dimensions) without running out of GPU memory.
Key observations:

- The method achieves perfect correctness on both GloVe datasets.
- The Pubs dataset shows non-zero error counts due to ties and floating-point differences in the reference implementation.
- Throughput increases with the number of queries, as the GPU is better utilised for larger batch sizes.
- The runtime scales linearly with the number of queries, as expected for a brute-force method.

Overall, the GPU brute-force implementation provides a reliable and efficient ground-truth baseline for evaluating the approximate nearest neighbour methods in the subsequent parts of the assignment.